

ui.editor 全面详解

`ui.editor` 是 NiceGUI 基于 Quasar QEditor 封装的所见即所得（WYSIWYG）富文本编辑器组件，核心用于网页端快速构建文本格式化编辑功能，输出结果为 HTML 字符串，支持与其他组件数据绑定，适用于表单输入、内容编辑、文档撰写等场景。以下从核心特性、配置参数、使用方法、进阶用法等维度展开全面解析。

一、核心定位与基础特性

1. 核心功能

- 富文本格式化：支持字体样式（加粗、斜体、下划线）、对齐方式（左对齐、居中、右对齐）、列表（有序 / 无序列表）、缩进、链接插入、代码块等基础编辑功能。
- 数据双向绑定：编辑内容实时同步为 HTML 字符串，支持与其他组件（如 `ui.markdown`、`ui.label`）联动展示。
- 状态控制：支持启用 / 禁用编辑器、清空内容、动态设置编辑值。
- 样式定制：可通过 Tailwind CSS 类或自定义样式修改编辑器外观（如高度、边框、背景色）。
- 事件监听：支持内容变更事件回调，实时响应编辑操作。

2. 基础用法

通过 `ui.editor()` 创建富文本编辑器，可直接绑定内容到其他组件，示例代码结构如下：

```
from nicegui import ui

# 创建富文本编辑器，设置占位提示
editor = ui.editor(placeholder='请在此输入内容...')

# 绑定编辑器内容到 Markdown 组件，实时显示 HTML 源码
ui.markdown('### 编辑结果（HTML 源码）').bind_content_from(
    editor, 'value',
    backward=lambda html: f'```\n{html}\n```' or "<!-- 暂无内容 -->")

ui.run()
```

二、关键配置参数

`ui.editor` 的初始化参数简洁直观，主要用于设置初始值和内容变更回调，以下是完整参数说明：

参数名	类型	说明	默认值	注意事项
value	字符串	编辑器初始内容（支持 HTML 格式）	""	若传入 HTML 字符串，编辑器会自动解析为对应格式化文本

参数名	类型	说明	默认值	注意事项
on_change	回调函数	内容变更时触发的回调，参数为 <code>ValueChangeEventArguments</code>	-	回调参数 <code>e.value</code> 为当前编辑内容的 HTML 字符串
placeholder	字符串	无内容时的占位提示文本	""	仅在编辑器为空时显示

补充属性（可直接访问或修改）

属性名	类型	说明
enabled	布尔值	编辑器是否启用（ <code>True</code> 可编辑， <code>False</code> 只读）
visible	布尔值	编辑器是否可见
html_id	字符串	组件 DOM 元素的 ID（2.16.0 版本新增，用于自定义 CSS/JS 操作）

三、核心功能用法

1. 初始内容与动态赋值

(1) 设置初始 HTML 内容

编辑器支持直接传入 HTML 字符串作为初始值，自动解析为格式化文本：

```
from nicegui import ui

# 初始内容为带格式的 HTML 字符串
initial_html = '''
<h3>富文本编辑器示例</h3>
<p>这是一段 <strong>加粗文本</strong>，这是一段 <em>斜体文本</em>。</p>
<ul>
  <li>无序列表项 1</li>
  <li>无序列表项 2</li>
</ul>
<p><a href="https://nicegui.io">NiceGUI 官网链接</a></p>
'''

editor = ui.editor(value=initial_html, placeholder='请编辑内容...')

# 实时显示解析后的纯文本（去除 HTML 标签）
def strip_html(html: str) -> str:
    import re
    return re.sub(r'<[^\>]*>', '', html) if html else ''

ui.label('纯文本预览: ').bind_text_from(editor, 'value', backward=strip_html)

ui.run()
```

(2) 动态修改编辑器内容

通过 `editor.set_value()` 方法或直接赋值 `editor.value`，可动态更新编辑器内容：

```
from nicegui import ui

editor = ui.editor(placeholder='请编辑内容...')

with ui.row():
    # 按钮设置预设内容
    ui.button('插入示例文本', on_click=lambda: editor.set_value(
        '<p>动态插入的 <span style="color: red;">红色文本</span></p>'
    ))
    # 按钮清空内容
    ui.button('清空内容', on_click=editor.clear)

ui.run()
```

2. 内容变更监听

通过 `on_change` 参数或 `on_value_change()` 方法，监听编辑器内容变化：

```
from nicegui import ui
from nicegui.events import ValueChangeEventArguments

def on_content_change(e: ValueChangeEventArguments):
    # e.value 为当前内容的 HTML 字符串
    html_length = len(e.value) if e.value else 0
    ui.notify(f'内容已更新，HTML 长度: {html_length} 字符')

# 方式 1: 初始化时指定 on_change
editor = ui.editor(on_change=on_content_change)

# 方式 2: 后续绑定 on_value_change
# editor.on_value_change(on_content_change)

ui.run()
```

3. 启用 / 禁用编辑器

通过 `enabled` 属性或 `enable()/disable()` 方法，控制编辑器是否可编辑：

```

from nicegui import ui

editor = ui.editor(placeholder='请编辑内容...')

with ui.row():
    ui.button('启用编辑', on_click=editor.enable)
    ui.button('禁用编辑', on_click=editor.disable)
    # 开关绑定启用状态
    ui.switch('允许编辑', value=True).bind_value_to(editor, 'enabled')

ui.run()

```

4. 样式定制

通过 `classes` 或 `style` 参数，自定义编辑器的外观（如高度、边框、背景色）：

```

from nicegui import ui

# 自定义样式：设置高度、边框、圆角、内边距
editor = ui.editor(
    placeholder='请在此输入内容...',
    classes='h-64 border-2 border-gray-300 rounded-lg p-4', # Tailwind 类
    style='font-family: "Microsoft YaHei", sans-serif;' # 自定义 CSS 样式
)

ui.run()

```

四、进阶用法

1. 与表单结合使用

将编辑器作为表单字段，收集用户输入的富文本内容并提交：

```

from nicegui import ui

def submit_form():
    # 获取编辑器的 HTML 内容
    content = editor.value
    if not content:
        ui.notify('请输入内容后提交', type='warning')
        return
    # 模拟表单提交（实际场景可发送到后端存储）
    ui.notify('表单提交成功!', type='success')
    print('提交的 HTML 内容: ', content)

ui.label('表单内容编辑').classes('text-xl font-bold')
editor = ui.editor(placeholder='请输入表单内容...').classes('h-48')

with ui.row().classes('mt-4'):
    ui.button('提交表单', on_click=submit_form).classes('bg-blue-500 text-white')
    ui.button('重置内容', on_click=editor.clear)

```

```
ui.run()
```

2. 限制编辑功能（通过 Quasar 原生属性）

`ui.editor` 基于 Quasar QEditor，可通过 `props` 传递 Quasar 原生参数，限制部分编辑功能（如隐藏某些工具按钮）：

```
from nicegui import ui

# 仅保留基础编辑功能（加粗、斜体、列表、链接），隐藏其他工具
editor = ui.editor(
    placeholder='限制编辑功能的编辑器...',
    props={
        'toolbar': [
            'bold', 'italic', 'underline', # 字体样式
            '|', # 分隔线
            'left', 'center', 'right', # 对齐方式
            '|',
            'unordered-list', 'ordered-list', # 列表
            '|',
            'link' # 链接
        ]
    }
).classes('h-48')

ui.run()
```

3. 自定义工具按钮（通过插槽）

通过 `add_slot()` 方法添加自定义工具按钮，扩展编辑器功能（如插入预设模板、特殊字符）：

```
from nicegui import ui

editor = ui.editor(placeholder='支持自定义工具的编辑器...').classes('h-48')

# 添加自定义工具按钮：插入日期
editor.add_slot('toolbar-end', '''
    <q-btn
      label="插入日期"
      icon="event"
      size="sm"
      @click="$parent.$emit('insert', new Date().toLocaleDateString())"
    />
''')

# 添加自定义工具按钮：插入特殊符号
editor.add_slot('toolbar-end', '''
    <q-btn
      label="插入★"
      size="sm"
    />
''')
```

```
        @click="$parent.$emit('insert', '★')"
```

```
    />
```

```
'''
```

```
ui.run()
```

4. 内容预览与导出

结合 `ui.html` 组件实现富文本预览，或导出内容为 HTML 文件：

```
from nicegui import ui

editor = ui.editor(placeholder='编辑后可预览或导出...').classes('h-48')

# 富文本预览区域
ui.label('实时预览: ').classes('mt-4 font-bold')
preview = ui.html().bind_content_from(editor, 'value')

# 导出 HTML 文件
def export_html():
    content = editor.value or '<p>无编辑内容</p>'
    # 创建下载链接
    ui.download(
        content,
        filename='editor_content.html',
        mime_type='text/html'
    )

ui.button('导出 HTML 文件', on_click=export_html).classes('mt-4 bg-green-500 text-white')

ui.run()
```

五、核心方法速查表

方法	说明	参数	示例
set_value(value)	设置编辑器内容	value: HTML 字符串或纯文本	<code>editor.set_value('新内容')</code>
get_value()	获取当前编辑内容 (HTML 字符串)	-	<code>content = editor.get_value()</code>
clear()	清空编辑器内容	-	<code>editor.clear()</code>
enable()	启用编辑器 (可编辑)	-	<code>editor.enable()</code>
disable()	禁用编辑器 (只读)	-	<code>editor.disable()</code>
set_enabled(enabled)	动态设置启用状态	enabled: 布尔值 (True/False)	<code>editor.set_enabled(False)</code>
on_value_change(callback)	绑定内容变更回调	callback: 回调函数 (参数为 <code>ValueChangeEventArguments</code>)	<code>editor.on_value_change(lambda e: print(e.value))</code>
add_slot(name, template)	添加 Vue 插槽 (扩展工具条等)	name: 插槽名称; <code>template</code> : Vue 模板字符串	见「自定义工具按钮」示例

方法	说明	参数	示例
update()	刷新组件显示	-	动态修改属性后调用，确保界面同步

六、注意事项与最佳实践

1. 内容安全

- 编辑器输出为 HTML 字符串，若用于公开展示，需对内容进行 XSS 过滤（如使用 `bleach` 库清理危险标签和属性），避免恶意脚本注入。
- 示例：使用 `bleach` 过滤危险内容

```
import bleach
from nicegui import ui

def sanitize_html(html: str) -> str:
    # 允许的 HTML 标签和属性
    allowed_tags = ['h1', 'h2', 'h3', 'p', 'strong', 'em', 'u', 'li', 'a']
    allowed_attrs = {'a': ['href', 'target']}
    return bleach.clean(html, tags=allowed_tags, attributes=allowed_attrs)

editor = ui.editor(placeholder='输入内容会自动过滤危险标签...')
# 预览过滤后的内容
ui.html().bind_content_from(editor, 'value', backward=sanitize_html)

ui.run()
```

2. 兼容性

- 编辑器基于 Quasar QEditor，支持主流浏览器（Chrome、Firefox、Edge），部分旧浏览器（如 IE）可能存在功能兼容问题。
- 自定义插槽和原生属性时，需参考 Quasar QEditor 官方文档（[Quasar QEditor](#)），确保参数格式正确。

3. 性能优化

- 当编辑内容较长（如超过 10000 字符）时，建议关闭实时绑定预览，改为通过按钮触发预览，避免频繁 DOM 更新导致卡顿。
- 示例：手动触发预览

```
from nicegui import ui

editor = ui.editor(placeholder='长文本编辑...').classes('h-64')
preview = ui.html().classes('mt-4 p-2 border')

ui.button('触发预览', on_click=lambda: preview.set_content(editor.value))

ui.run()
```

总结

`ui.editor` 是 NiceGUI 中轻量且易用的富文本编辑组件，通过封装 Quasar QEditor 提供了基础的文本格式化功能，同时支持灵活的扩展和定制。其核心优势在于与 NiceGUI 生态的无缝集成，可快速实现内容编辑、数据绑定、预览导出等完整流程，适用于简单的网页表单、文档编辑、评论输入等场景。对于复杂需求（如多人协作、高级格式支持），可结合 Quasar 原生属性或第三方 HTML 编辑库进一步扩展。